Making a Local Coding Agent Unique

Notes for the <_FOUR_PIE project: a local agent on a 4 GB RTX 3050 (deepseek-r1:1.5b planner, qwen2.5-coder:3b coder).

The core idea

Uniqueness for a project like this rarely comes from the architecture. It comes from picking a narrow thing and doing it better than anyone else, and the project's constraints already point to a few good candidates. No survey of other local agents has been done, so check what exists before committing to a direction.

Ideas that fit what you've already built

1. Own the "4 GB VRAM" niche. Most local agent projects assume 8-24 GB or quietly rely on a cloud model. If "works well on 4 GB" is the headline, there is a clear audience: people with old laptops, a 3050 or 1650, or a work machine. To back it up, treat VRAM as a first-class budget. Show per-stage VRAM and load times in the sidebar, and have the agent pick models automatically based on free VRAM.

2. Make the pipeline visible and honest. The trace line with red SKIPPED stages is already unusual, because most agents hide what they did. Go further with a run report after each task: what ran, what each model saw, what failed, what was unverified. "An agent that never claims more than it verified" is a real selling point, especially with small models that confidently produce wrong output.

3. Benchmark-driven model selection. `bench.py` is the most distinctive asset. Turn it into a feature: on first launch, the tool benchmarks whatever models the user has installed and assigns planner and coder roles from measured scores, not guesses. Publishing a leaderboard of "best model pairs per VRAM size" (4, 6, 8 GB) would make the project useful to people who never run the agent.

4. Verification by execution. Run the code, run the tests, feed real errors back. Small-model agents mostly skip this. Adding it and publishing the before and after pass rate gives a measurable claim.

5. Safety-first editing. The staging folder is a good start. Add a git checkpoint before every save and a one-command undo. Many hobby agents write straight into your files, so "never touches your files without a diff and an undo" is a trust feature.

6. A specific specialty. Rather than a general coder, make it excellent at one job: for example single-file scripts, small web pages, or "explain and refactor this folder". A 3B model that does one job reliably beats one that attempts everything.

Recommended combination

Combine ideas 1, 3 and 4 into one story: *a coding agent that fits in 4 GB, picks its own models by measuring them, and verifies its work by running it.* Each part already exists in the project or is a small step away, and together they form a clear identity.

What not to do

- Add features that other agents already do better, like plugin systems, many model providers, or a big tool loop.
- Chase a bigger UI first. Textual polish matters less than the claims you can prove.
- Call it unique before checking. Spend an hour searching GitHub for "local coding agent Ollama low VRAM" and read the top results. That shows what is crowded and where the gap is.

Concrete next step

Run `python bench.py --coexist` on the real GPU and write down the numbers. A README with real measurements on a 3050 already sets the project apart from most that only describe what they do.